Question 3

(20 points) Please create the Python simulation code for the modified TTF system described in Problem 1, run 100 replications, and record the time of system failure, denoted by Y1, Y2, . . . , Y100, and the average number of functional component, denoted by $\bar{S}$ 1, $\bar{S}$ 2, . . . , $\bar{S}$ 100. Then, calculate the 95% confident intervals for expected time of system failure and average number of functional component.

Base code

made changes to make it modular

In [129…

```python
# Libraries

import random
import math
import numpy as np
```

In [130…

```python
class TTF_Sim:

    def __init__(self, replication, failure_time, repair_time, criteria_t=None, seed=123):
        random.seed(seed) # setting random seed

        self.failure_time = failure_time
        self.repair_time = repair_time
        self.t = criteria_t # if stopping criteria is time and not end of system failure


        self.y_lst = [] # place holder for average time to system failure in each iteration
        self.s_lst = [] # place holder for average number of function component in each iteration

        self.replication = replication # number of replications

        for _ in range(self.replication): # iterate through n replications
            self.NextFailure = math.ceil(6*random.random()) # random time to failure
            self.NextRepair = float('inf') # set initial repair time to inf
```

```
            self.S = 3 # number of machines/states
            self.Slast = self.S # previous state
            self.Clock = 0 # initialize clock
            self.Tlast = self.Clock # previous clock
            self.Area = 0 # s(x_i-1)*(x_i - x_i-1)

            #choosing stopping criteria
            if self.t:
                self.criteria_t()
            else:
                self.criteria_s()

            self.s_lst.append(self.Area/self.Clock) # append avg. functional component
            self.y_lst.append(self.Clock) # append time to system failure
        # statistics

        for lst, stat in zip([self.s_lst, self.y_lst], ['avg. # functional component', 'time to system failure']):

            print(f'{stat} stats')
            print(f'count: {len(lst)}')
            print(f'max: {np.max(lst):.3f} | min: {np.min(lst):.3f}')
            print(f'sample mean: {np.mean(lst):.3f} | sample standard dev.: {np.std(lst, ddof=1):.3f}')
            z_critical = 1.96  # for 95% CI
            margin_error_z = z_critical * (np.std(lst, ddof=1) / np.sqrt(len(lst)))
            ci_lower_z = np.mean(lst) - margin_error_z
            ci_upper_z = np.mean(lst) + margin_error_z
            print(f"95% confidence interval: ({ci_lower_z:.3f}, {ci_upper_z:.3f})")

            print('-----------------------------------------------------------')

    # Timer
    def Timer(self):
        '''
        Function to check if event is a failure/repair
        '''

        if self.NextFailure < self.NextRepair:
            NextEvent = 'Failure'
            self.Clock = self.NextFailure
            self.NextFailure = float('inf')
        elif self.NextFailure >= self.NextRepair:
            NextEvent = 'Repair'
```

```
            self.Clock = self.NextRepair
            self.NextRepair = float('inf')
        return NextEvent

    # Repair Event
    def Repair(self):
        self.S += 1 # increase state by 1

        if  self.S in [1, 2]:
            self.NextRepair = self.Clock + self.repair_time() # update repaire time
        if self.S == 1:
            self.NextFailure = self.Clock + self.failure_time() # update failure time

            self.Area = self.Area + self.Slast * (self.Clock - self.Tlast) # accumulate area
            self.Tlast = self.Clock # set prev clock
            self.Slast = self.S # set prev state

    # Failure Event
    def Failure(self):
        '''
        variable updates, when event is failure
        '''

        self.S -= 1 # decrease state by 1

        if self.S >= 1:
            self.NextFailure = self.Clock + self.failure_time() # update failure time
        if self.S == 2:
            self.NextRepair = self.Clock + self.repair_time() # update repaire time

        self.Area = self.Area + self.Slast * (self.Clock - self.Tlast) # accumulate area
        self.Tlast = self.Clock # set prev clock
        self.Slast = self.S # set prev state

    # Stopping criteria: S > 0
    def criteria_s(self):
        while self.S > 0: # stopping criteria
                self.NextEvent = self.Timer() # get next event; Failure/Repair

                if self.NextEvent == 'Failure': # call event
                    self.Failure()
                else:
```

```
                    self.Repair()

    # stopping criteria: some time t
    def criteria_t(self):
        while self.Clock <= self.t:
                self.NextEvent = self.Timer() # get next event; Failure/Repair

                if self.NextEvent == 'Failure': # call event
                    self.Failure()
                else:
                    self.Repair()
```

Part A

constant repair time, but random failure time

In [131…

```
# random failure time; uniform distribution U: [1, 6]
def failure_time():
    return random.randint(1, 6)

# fixed repair time
def repair_time():
    return 3.5

replication = 100

TTF_Sim(replication, failure_time, repair_time)
```

```
avg. # functional component stats
count: 100
max: 2.556 | min: 1.088
sample mean: 1.726 | sample standard dev.: 0.327
95% confidence interval: (1.662, 1.790)
----------------------------------------------------------
time to system failure stats
count: 100
max: 217.000 | min: 3.000
sample mean: 40.460 | sample standard dev.: 39.990
95% confidence interval: (32.622, 48.298)
----------------------------------------------------------
```

Out[131… <__main__.TTF_Sim at 0x2515ffb4980>

Part B.a

stopping criteria is clock <= 1000; with 100 replication

In [132…

```
# random failure time; uniform distribution U: [1, 6]
def failure_time():
    return random.randint(1, 6)

# fixed repair time
def repair_time():
    return 3.5

replication = 100

# setting stopping criteria as clock < 1000; using argument criteria_t
TTF_Sim(replication, failure_time, repair_time, criteria_t=1000)
```

```
avg. # functional component stats
count: 100
max: 1.280 | min: 1.001
sample mean: 1.163 | sample standard dev.: 0.060
95% confidence interval: (1.151, 1.174)
----------------------------------------------------------
time to system failure stats
count: 100
max: 1003.500 | min: 1000.500
sample mean: 1001.430 | sample standard dev.: 0.916
95% confidence interval: (1001.251, 1001.609)
----------------------------------------------------------
```

Out[132… <__main__.TTF_Sim at 0x2515ff120d0>

Part B.b

repair time is either 1.25 or 2.75

In [133…

```python
# random failure time; uniform distribution U: [, 6]
def failure_time():
    return random.uniform(1.0, 6.0)

# choose 1.25 or 2.75 as repair time
def repair_time():
    return random.choice([1.25, 2.75])

replication = 100

TTF_Sim(replication, failure_time, repair_time)
```

```
avg. # functional component stats
count: 100
max: 2.427 | min: 1.557
sample mean: 1.808 | sample standard dev.: 0.113
95% confidence interval: (1.786, 1.830)
----------------------------------------------------------
time to system failure stats
count: 100
max: 1819.452 | min: 6.411
sample mean: 362.505 | sample standard dev.: 341.831
95% confidence interval: (295.506, 429.504)
----------------------------------------------------------
```

Out[133… <__main__.TTF_Sim at 0x2515ff12ad0>